

L10 — Merge Sort: Merging Arrays and Complexity Analysis

Unit: 1 | Chapter: Sorting Techniques | BT Level: BT4 | CO: CO1, CO5

Learning Objectives

- Implement the merge sort algorithm using divide-and-conquer
 - Implement the merge subroutine for combining two sorted arrays
 - Derive the time complexity using the recurrence relation
 - Compare merge sort with quick sort
-

1. Core Idea

Merge Sort is a **divide-and-conquer** algorithm:

1. **Divide:** Split the array into two halves
2. **Conquer:** Recursively sort each half
3. **Combine:** Merge the two sorted halves into one sorted array

The key insight: merging two sorted arrays of sizes m and n takes $O(m + n)$ time.

2. The Merge Operation

The merge step combines two sorted sub-arrays into a single sorted array.

```
def merge(arr, left, mid, right):  
    """  
    Merge arr[left..mid] and arr[mid+1..right] – both already sorted.  
    """  
    # Sizes of the two sub-arrays  
    n1 = mid - left + 1  
    n2 = right - mid  
  
    # Create temporary arrays  
    L = arr[left : left + n1]    # O(n1) space  
    R = arr[mid + 1 : mid + 1 + n2] # O(n2) space  
  
    # Merge back into arr[left..right]  
    i = 0        # Index for L  
    j = 0        # Index for R  
    k = left     # Index for arr  
  
    while i < n1 and j < n2:  
        if L[i] <= R[j]:        # <= ensures stability  
            arr[k] = L[i]
```

```

        i += 1
    else:
        arr[k] = R[j]
        j += 1
    k += 1

# Copy remaining elements
while i < n1:
    arr[k] = L[i]
    i += 1
    k += 1

while j < n2:
    arr[k] = R[j]
    j += 1
    k += 1

```

3. Merge Sort Algorithm

```

def merge_sort(arr, left, right):
    """
    Sort arr[left..right] using merge sort.
    """
    if left < right:          # Base case: single element is already sorted
        mid = (left + right) // 2
        merge_sort(arr, left, mid)          # Sort left half
        merge_sort(arr, mid + 1, right)     # Sort right half
        merge(arr, left, mid, right)        # Merge sorted halves

# Usage
arr = [38, 27, 43, 3, 9, 82, 10]
merge_sort(arr, 0, len(arr) - 1)
print(arr)  # [3, 9, 10, 27, 38, 43, 82]

```

4. Recursive Decomposition Trace

Array: [38, 27, 43, 3, 9, 82, 10]

```

          [38, 27, 43, 3, 9, 82, 10]
              /           \
        [38, 27, 43, 3]   [9, 82, 10]
            /     \       /     \
        [38, 27] [43, 3] [9, 82] [10]
          /  \   /  \   /  \
        [38] [27] [43] [3] [9] [82]

```

Merge up:

```

[27, 38] [3, 43]   [9, 82] [10]
  [3, 27, 38, 43]   [9, 10, 82]
    [3, 9, 10, 27, 38, 43, 82]

```

5. Merge Step Trace

Merging [3, 27, 38, 43] and [9, 10, 82]:

Step	L[i]	R[j]	arr[k]	i	j	k
1	3	9	3	1	0	0
2	27	9	9	1	1	1
3	27	10	10	1	2	2
4	27	82	27	2	2	3
5	38	82	38	3	2	4
6	43	82	43	4	2	5
7	—	82	82	4	3	6

Result: [3, 9, 10, 27, 38, 43, 82] ✓

6. Complexity Analysis

6.1 Recurrence Relation

At each level of recursion:

- **Divide:** $O(1)$ — compute midpoint
- **Conquer:** 2 recursive calls on $n/2$ size each $\rightarrow 2T(n/2)$
- **Combine (merge):** $O(n)$ — merge two sorted halves

$$T(n) = 2T(n/2) + O(n)$$

6.2 Solving by Master Theorem

For $T(n) = aT(n/b) + O(n^d)$:

- $a = 2, b = 2, d = 1$
- $\log_b(a) = \log_2(2) = 1 = d$

Case 2: $T(n) = O(n^d \cdot \log n) = O(n \log n)$

6.3 Recursion Tree Method

Level	Sub-problems	Size each	Work per level
0	1	n	$O(n)$
1	2	$n/2$	$2 \times O(n/2) = O(n)$
2	4	$n/4$	$4 \times O(n/4) = O(n)$
k	2^k	$n/2^k$	$O(n)$

Levels: $\log_2(n)$, each costing $O(n)$

Total: $O(n \log n)$ ✓

6.4 Summary Table

Case	Time	Space
Best	$O(n \log n)$	$O(n)$
Average	$O(n \log n)$	$O(n)$
Worst	$O(n \log n)$	$O(n)$

Space: $O(n)$ auxiliary (for the temporary arrays in merge)

7. Properties

Property	Value
Time (all cases)	$O(n \log n)$
Space	$O(n)$
In-place	No
Stable	Yes
Adaptive	No

Stability: The condition $L[i] \leq R[j]$ (not strictly $<$) ensures equal elements from the left sub-array come first $\rightarrow$ stable.

8. Bottom-Up Merge Sort (Iterative)

Avoids recursion entirely — useful when stack depth is a concern:

```
def merge_sort_bottom_up(arr):
    n = len(arr)
    size = 1
    while size < n:
        for left in range(0, n, 2 * size):
            mid = min(left + size - 1, n - 1)
            right = min(left + 2 * size - 1, n - 1)
            if mid < right:
                merge(arr, left, mid, right)
        size *= 2
    return arr
```

Time: $O(n \log n)$ | **Space:** $O(n)$

9. Use Cases for Merge Sort

- **External sorting:** When data doesn't fit in memory (merge sorted chunks from disk)
- **Sorting linked lists:** Merge sort works naturally on linked lists (no random access needed)
- **Stable sort required:** When preserving relative order of equal elements matters
- **Guaranteed $O(n \log n)$:** Unlike quick sort, merge sort has no worst-case degradation

10. Merge Sort vs Quick Sort

Property	Merge Sort	Quick Sort
Worst case	$O(n \log n)$	$O(n^2)$
Average case	$O(n \log n)$	$O(n \log n)$
Space	$O(n)$	$O(\log n)$
Stable	Yes	No
Cache performance	Moderate	Excellent
Use for linked lists	Yes	No

Summary

- Merge sort: divide into halves → recursively sort → merge.
 - Merge step is $O(n)$: compare element-by-element from two sorted halves.
 - Recurrence $T(n) = 2T(n/2) + O(n) \rightarrow \mathbf{O(n \log n)}$ in all cases.
 - Requires $O(n)$ extra space but guarantees stability and consistent performance.
-

References

- Cormen et al., *Introduction to Algorithms*, Ch. 2.3, Ch. 4 (MIT Press, 2022)
- Skiena, *The Algorithm Design Manual*, Ch. 4.5 (Springer, 2020)
- Laaksonen, *Guide to Competitive Programming*, Ch. 3.1 (Springer, 2024)